

Experiment - 02.

- AIM: Write a program to implement parallel Bubble sort and merge sort using Open MP. Use existing algorithms and measure the performance of sequential and parallel algorithms.

- THEORY:

Sorting is a fundamental operation in Computer science and plays a vital role in data processing, searching and optimization problems. With the availability of multicore processors, parallelizing traditional algorithms has become essential to improve execution speed.

(1). Parallel programming and OpenMP -

- Parallel programming allows multiple parts of a program to execute simultaneously. In shared memory systems, all threads operate on a common memory space.
- OpenMP is a widely used API that supports parallel programming by using compiler directives. It enables parallel execution with minimal modifications to existing sequential programs.

(2) BUBBLE SORT -

Sequential bubble sort -

- Bubble sort repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order.
- After each pass, the largest element moves to end of array.

Example - Unsorted array - $[5, 1, 4, 2]$

Pass 1 : $(5, 1) \rightarrow \text{swap}$ $[1, 5, 4, 2]$

$(5, 4) \rightarrow \text{swap}$ $[1, 4, 5, 2]$

$(5, 2) \rightarrow \text{swap}$ $[1, 4, 2, 5]$

Pass 2 : $(1, 4) \rightarrow \text{no swap}$.

$(4, 2) \rightarrow \text{swap} \rightarrow [1, 2, 4, 5]$.

Sorted array - $[1, 2, 4, 5]$.

(3) Parallel Bubble sort (Odd-even Transposition method).

- Bubble sort is parallelized using odd-even transposition sort, where comparisons are divided into two independent phases.
 - Odd phase - Compare elements at odd even indices $(1, 2), (3, 4), (5, 6) \dots$
 - Even phase - Compare elements at even - odd indices $(0, 1), (2, 3), (4, 5) \dots$
- Each phase can be executed in parallel using Open MP threads.

Initial array - $[5, 1, 4, 2, 8, 3]$

Even Phase - Compare $(0,1), (2,3), (4,5)$

→ $[1, 5, 2, 4, 3, 8]$

Odd Phase : Compare $(1,2), (3,4)$

→ $[1, 2, 5, 3, 4, 8]$

Repeat until sorted

(4) Merge Sort -

a) Sequential merge sort -

1. Merge sort follows the divide and conquer strategy -
2. Divide the array into two halves.
3. Recursively sort both halves.
4. Merge the sorted halves.

Example - $[38, 27, 43, 3]$

Division - $[38, 27]$ $[43, 3]$

Merging - $[27, 38]$ $[3, 43]$

Final merge - $[3, 27, 38, 43]$

b) Parallel Merge Sort -

Merge sort is naturally suitable for parallel execution because recursive calls for left and right subarrays are independent.

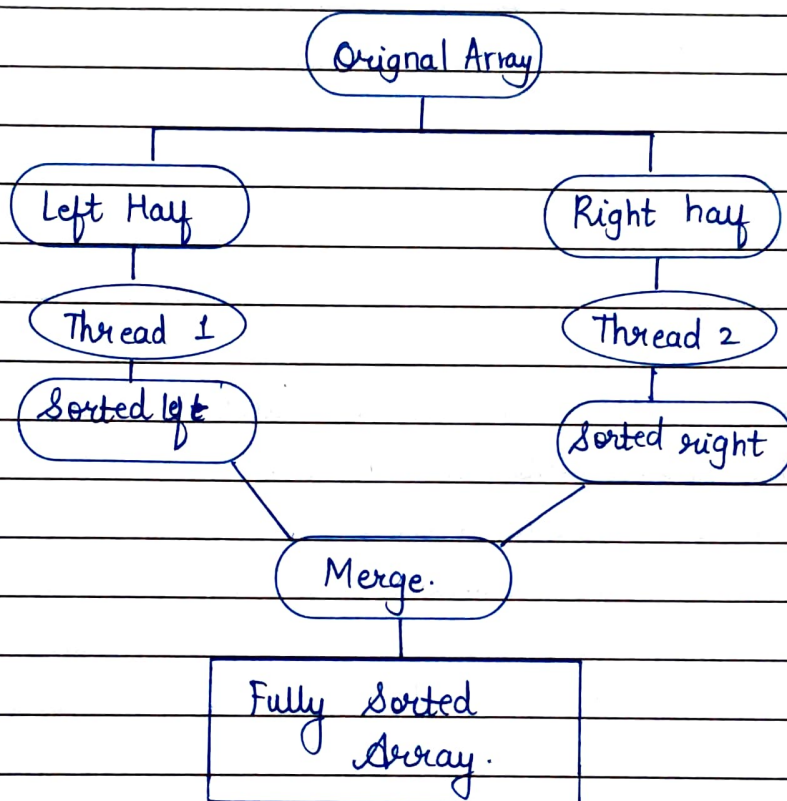
Parallel Approach - left half sorting - Thread 1

- Right half sorting - Thread 2

- Merge steps performed after synchronization.

OPENMP allows these recursive calls to execute concurrently using parallel sections or tasks.

* Diagrammatic representation of Parallel Merge Sort -



(5) Execution Time Analysis -

Sequential vs. Parallel Execution -

- ° In sequential algorithm, all operations are executed one after another using single processor core. Hence total execution time depends entirely on number of operations and efficiency of algorithm.
- ° In parallel algorithm, the workload is divided among multiple threads that execute concurrently.

Ideally this reduces the total execution time, however practical factors such as thread creation, synchronization and memory access overhead influence the actual performance.

(6) Execution Time Behaviour of Bubble sort -

- Sequential bubble sort performs a large number of comparisons and swaps, leading to $O(n^2)$ time complexity. As the input size increases, the execution time grows rapidly.
- In parallel bubble sort (odd even transposition):
 - Multiple comparisons are executed simultaneously
 - Each phase (odd or even) requires synchronization
 - The algorithm still performs $O(n^2)$ operations.
- As a result, the reduction in execution time is marginal, especially for small and medium-sized arrays. The overhead of synchronization often limits the achievable speedup.

(7) Execution time and behaviour of Merge Sort -

- Sequential merge sort operates with $O(n \log n)$ complexity. The divide and conquer nature allows it to efficiently handle large datasets.
- In parallel merge sort:
 - Recursive calls for left and right subarrays execute concurrently.

- Workload is evenly distributed among threads.
- Synchronization is required only during the merge phase.
- This significantly reduces execution time, particularly for large input sizes.

(8) Impact of Number of Threads -

- Increasing the number of threads reduces execution time up to a certain limit.
- Beyond that limit, performance improvement saturates due to:
 - Synchronization cost.
 - Memory contention.
 - Limited parallel regions.
- Merge Sort benefits more than Bubble sort from additional threads.

* Conclusion:

Thus we have successfully executed parallel bubble sort and merge sort and hence analyzed their performance accordingly.

* * * * *